

A Multi-Agent Reinforcement Learning Algorithm Embedded with Opponent Modeling

Shun Li^{a,c}, Xiao Su^a and Bing Shi^{a,b,*}

^aSchool of Computer Science and Artificial Intelligence, Wuhan University of Technology, Wuhan 430070, China

^bShenzhen Research Institute of Wuhan University of Technology, Shenzhen 518000, China

^cCollege of Intelligence Science and Technology, National University of Defense Technology, Changsha 410072, China

Abstract. In multi-agent systems, how to cooperate with each other to jointly confront opponent agents is a very important issue (i.e., multi-agent cooperative confrontation problem). In recent years, multi-agent reinforcement learning has developed rapidly and become a general paradigm for solving multi-agent decision-making problems, so it has naturally been used to solve multi-agent cooperative confrontation problems. However, existing works usually focus on the cooperation among multiple agents, making it difficult for agents to adapt to highly dynamic adversarial environments. To address this issue, in this paper, we propose a multi-agent reinforcement learning algorithm embedded with opponent modeling (**MARLeOM**). The opponent modeling module constructs multi-level opponent models using the environment model and recursive reasoning, and then mixes multi-level opponent models to enhance the representational capability. The multi-agent reinforcement learning part adopts *CTDE* mechanism and *Actor-Critic* framework. In the centralized training phase, Critic can obtain the observation and action information of all agents to guide the learning of Actor; in the decentralized execution phase, Actor makes decisions based on local information, which includes its own observation and the predicted opponent action based on the opponent's observation using the opponent modeling module. The combination of opponent modeling and multi-agent reinforcement learning enables agents to learn optimal cooperative confrontation methods. Empirical experiments on multiple cooperative adversarial tasks demonstrate that **MARLeOM** can achieve more effective adaptation and better performance than baseline methods.

1 Introduction

Multi-agent systems, characterized by their constituent agents' abilities to perceive, communicate, decide, and act, are prevalent in various real-world scenarios, including team sports, board games, airport security, and military operations [10]. These systems often entail intricate interactions, blending cooperation among allies with competition against adversaries. A prime example is found in multiplayer online games and military simulations, where opposing sides comprise autonomous agents engaging in both collaborative and adversarial behaviors. We denote such contexts as multi-agent cooperative confrontation problems, which are not only ubiquitous but also

present significant challenges for research in optimal strategy design and learning.

Reinforcement learning (RL), a subset of machine learning, involves agents interacting continuously with their environment to refine their policies through experience [30]. RL is pivotal to advancing toward general artificial intelligence [28]. Multi-agent reinforcement learning (MARL) extends this paradigm to multi-agent systems, offering a universal approach to real-time decision-making that surpasses traditional control methods. The integration of deep neural networks has further propelled MARL's applicability to complex, high-dimensional state spaces, establishing it as a leading technique in fields such as cooperative multi-UAV operations [7], robotic sports [5], and computational resource management [23].

Despite the prowess of MARL in addressing decision-making within multi-agent systems, existing research predominantly emphasizes agent coordination, often overlooking the competitive dynamics essential for adaptation in highly dynamic adversarial environments. To bridge this gap, we introduce **MARLeOM**, a novel algorithm that fuses opponent modeling with multi-agent reinforcement learning, aiming to enhance agents' capabilities in adversarial contexts.

Our main contributions are summarized as follow:

- We propose a learning method in a mixed cooperative-competitive environment, in which each agent can efficiently learn not only the cooperative strategies with teammates but also competitive behaviors to confront the adversaries.
- We empirically evaluate our proposed **MARLeOM** algorithm on three mixed cooperative and competitive environments and the agents trained by **MARLeOM** outperform baseline policies on all these environments.

In the rest of the paper, we will firstly present related works in Section 2. Problem definition and the basic algorithm are described in Section 3. Our main algorithm, **MARLeOM**, is introduced in Section 4. Experimental results are in Section 5. Finally, Section 6 summarizes this paper and the future work.

2 Related Work

2.1 Multi-Agent Games

In instances where the underlying dynamics are well understood or the quantity of data gathered approaches infinity, extensive research

* Corresponding Author. Email: bingshi@whut.edu.cn

efforts have been directed towards the study of learning Nash equilibria in Markov games [21, 18, 34]. When the dynamics of Markov games are unknown, certain studies have implemented self-play algorithms that integrate exploration strategies to identify Nash equilibria [33, 3, 2, 37, 22]. In cases where the algorithm can only control a single participant within an adversarial environment, the R-max algorithm has been proposed in [4], which is capable of approximating the value of Nash equilibrium. Subsequent research advancements [33, 32, 19] have also demonstrated similar or superior performance when compared to the value of Nash equilibrium.

2.2 Multi-Agent Reinforcement Learning

In the early stages of MARL research, two predominant learning frameworks emerged: centralized learning and decentralized learning. Centralized learning [6] treats all agents as a single entity, aiming to directly learn a unified global policy. In contrast, decentralized learning [21] allows each agent to independently learn its own local policy. The challenge with centralized learning primarily stems from the exponential growth of the state-action space, while decentralized learning grapples with the issue of environmental non-stationarity. Subsequently, the method of Centralized Training with Decentralized Execution (CTDE) was introduced, which amalgamates the strengths of both centralized and decentralized learning, effectively mitigating the aforementioned challenges. CTDE methods can be categorized into two main types based on their core architecture: one [24, 11] that employs the actor-critic paradigm and another that is based on value decomposition [29, 27]. The development of these methods signifies a significant advancement in the MARL field towards addressing complex interactive challenges.

2.3 Opponent Modeling

Opponent modeling in MARL enhances policy robustness by integrating insights into adversary behaviors. Simple models are inefficient, so embedding vectors represent policies, as in [13]. Techniques like DRON [15] and DPIQN [17] predict opponent actions, while ToMnet [25] and SOM [26] foster theory of mind. Recursive reasoning in repeated games, as in R2-B2 [8], accelerates convergence. PR2 [36] and GR2 [35] apply joint Q-functions for reasoning. LOLA [12] and Meta-PG [1] adapt to learning adversaries, with Meta-MAPG [20] considering policy impacts on future opponents.

3 Background

In this section, we describe our problem formulation and the basic algorithm.

3.1 Partially Observable Markov Game

In addressing the multi-agent decision-making problem within a cooperative-competitive mixed environment, the problem is commonly abstracted as a Partially Observable Markov Game (POMG). A POMG can be formalized as a tuple $G = \langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{N} = \{1, 2, \dots, N\}$ denotes the set of indices for the agents. At each discrete time step t , the environment generates a global state $s \in \mathcal{S}$, and each agent $i \in \mathcal{N}$ makes decisions based on its own local observation $o^i \in \mathcal{O}^i$. The joint observation of all agents is represented as $\mathbf{o} = \langle o^1, o^2, \dots, o^N \rangle$, with the corresponding joint observation space being $\mathcal{O} = \prod_{i=1}^N \mathcal{O}^i$. Subsequently, agent i takes an action $a^i \in \mathcal{A}^i$, and the joint action of all agents is formed

as $\mathbf{a} = \langle a^1, a^2, \dots, a^N \rangle$, with the joint action space defined as $\mathcal{A} = \prod_{i=1}^N \mathcal{A}^i$. After the agents perform their actions, the environment transitions to the next state s' according to the state transition function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$, where $\Delta(\mathcal{S})$ denotes the probability distribution over the state space $\mathcal{S}$. Each agent i receives an immediate reward r^i based on the reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, and the joint reward of all agents is expressed as $\mathbf{r} = \langle r^1, r^2, \dots, r^N \rangle$. Over a horizon of T time steps, the objective of agent i is to maximize its cumulative discounted return $R^i = \sum_{t=1}^T \gamma^{t-1} r_t^i$, where the discount factor $\gamma \in [0, 1]$ is used to balance the trade-off between immediate and future rewards.

In the context of multi-agent cooperative confrontation problems, we consider a scenario involving n teammate agents and m opponent agents ($n + m = N$). The action and reward of the teammate agent are still denoted as $a^i \sim \pi^i(\cdot|o^i)$, r^i respectively. For the sake of research convenience, we treat all opponent agents as a single collective opponent¹, whose joint action is represented as $\mathbf{a}^o \sim \pi^o(\cdot|o^o)$, and the reward is represented as r^o .

3.2 CTDE and MADDPG

The CTDE framework is a prevalent approach in MARL, designed to address the challenges encountered when training policies in multi-agent environments. The core concept of the CTDE framework is the separation of the training process from the execution process. During the training phase, all agents participate in a centralized decision-making process, which enables them to learn global information and cooperative strategies. However, during the actual execution of policies, each agent acts independently, making decisions based on local observations, which aligns with the typical requirement for autonomous operation in multi-agent systems.

The MADDPG [24] algorithm is a typical implementation within the CTDE framework. This algorithm extends the DDPG algorithm and is designed to address multi-agent cooperation and competition problems with continuous action spaces. Specifically, in a game setting involving N agents, for each agent i , its deterministic policy is denoted as μ^i , with the corresponding policy network parameters being θ^i . The policy gradient of the expected return for agent i with respect to the policy parameters can be derived as follows:

$$\nabla_{\theta^i} J(\theta^i) = \mathbb{E}_{\mathbf{o} \sim \mathcal{D}} \left[\nabla_{\theta^i} \mu^i(o^i) \cdot \nabla_{\mathbf{a}^i} Q^i(\mathbf{o}, a^1, \dots, a^N) \Big|_{\mathbf{a}^i = \mu^i(o^i)} \right]. \quad (1)$$

Here, $\mathcal{D}$ represents the experience replay pool, which stores data in the format of $\langle \mathbf{o}, \mathbf{a}, \mathbf{o}', \mathbf{r} \rangle$, and Q^i denotes the global action-value function, encompassing the observations and actions of all agents. The update of the global action-value function employs target network techniques to minimize the following loss function:

$$\mathcal{L}(w^i) = \mathbb{E}_{\mathbf{o}, \mathbf{a}, \mathbf{o}', \mathbf{r}} \left[\left(Q^i(\mathbf{o}, a^1, \dots, a^N) - y^i \right)^2 \right], \quad (2)$$

$$y^i = r^i + \gamma Q^{i'}(\mathbf{o}', a^{1'}, \dots, a^{N'}) \Big|_{\mathbf{a}^{j'} = \mu^{j'}(o^j)}.$$

In this context, $Q^{i'}$ and $\mu^{j'}$ represent the global action-value function and policy of the target network, respectively. Their parameters are updated with a delay to enhance the stability of the training process.

¹ In this paper, we assume that $m < 10$.

4 Method

The MARLeOM architecture is based on the CTDE framework of multi-agent reinforcement learning, which is designed for inter-group coordination. And to enhance intra-group competition, opponent modeling is embedded into multi-agent reinforcement learning. The overall MARLeOM architecture is illustrated in Figure 1.

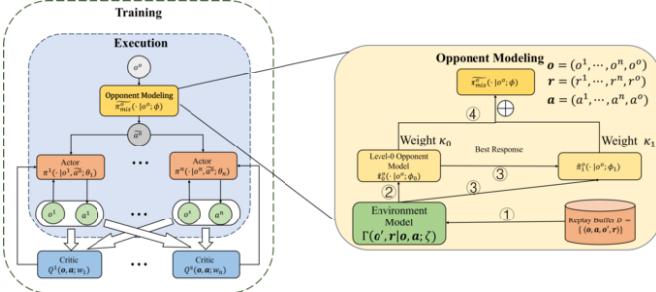


Figure 1: The MARLeOM architecture

4.1 The Opponent Modeling Module

In investigating the interactive mechanisms within multi-agent systems, we have adopted a data-driven modeling approach. Initially, by collecting interaction samples between agents, we construct an environment model. Within this environment model, we perform recursive logical reasoning to predict the strategic choices of the opponent and distill a set of opponent behavior models. To enhance the representational and generalization capabilities of these opponent behavior models, we apply the principles of Bayesian inference to perform a weighted fusion of multiple opponent behavior models, thereby forming a comprehensive mixed opponent model. This model is capable of dynamically adjusting the weights to adapt to the ever-evolving strategies of the opponent.

In the initial phase, our teammate agents employ a random strategy to interact with the opponent agent within a specific environment, collecting the experience data $\langle \mathbf{o}, \mathbf{a}, \mathbf{o}', \mathbf{r} \rangle$ generated from these interactions, and subsequently construct the experience replay pool $\mathcal{D}$. Here, $\mathbf{o} = \langle o^1, \dots, o^n, o^o \rangle$ denotes the observation states, $\mathbf{a} = \langle a^1, \dots, a^n, a^o \rangle$ represents the action set, and $\mathbf{r} = \langle r^1, \dots, r^n, r^o \rangle$ signifies the immediate rewards. We abstract the environment model as $\Gamma(\mathbf{o}', \mathbf{r} | \mathbf{o}, \mathbf{a}; \zeta)$ and train the environment model by minimizing the mean squared error (MSE) based on the data from the experience replay pool $\mathcal{D}$:

$$\mathbb{E}_{\mathbf{o}, \mathbf{a}, \mathbf{o}', \mathbf{r} \sim \mathcal{D}} \frac{1}{2} \|\mathbf{o}' - \hat{\mathbf{o}}'\|^2 + \frac{1}{2} \|\mathbf{r} - \hat{\mathbf{r}}\|^2, \quad (3)$$

where $\hat{\mathbf{o}}', \hat{\mathbf{r}}$ are the predicted next observation and reward by the model Γ , respectively. Concurrently, we utilize the experience data $\langle \mathbf{o}^o, \mathbf{a}^o \rangle$ to perform maximum likelihood estimation on the level-0 opponent model $\tilde{\pi}_0^o(\cdot | \mathbf{o}^o; \phi_0)$:

$$\mathbb{E}_{\mathbf{o}^o, \mathbf{a}^o \sim \mathcal{D}} \log \tilde{\pi}_0^o(\mathbf{a}^o | \mathbf{o}^o; \phi_0). \quad (4)$$

In practical scenarios, the opponent also exhibits learning capabilities, capable of estimating changes in the external environment and adjusting its behavior to maximize its own interests. To simulate this learning process, we employ the rollout [31] algorithm to infer the optimal response of the opponent to our agent's actions. Specifically, for the opponent's action a_t^o at time t , we uniformly sample the opponent's action sequences for the next k time steps and simulate the corresponding trajectories through the environment model. Among all simulated trajectories, we select the one with the highest

cumulative reward to determine the opponent's best response action a_t^{o*} :

$$a_t^{o*} = \operatorname{argmax}_{a_t^o} \max_{a_{t+1}^o, \dots, a_{t+k}^o \sim \text{Unif}} \sum_{j=0}^k \gamma^j r_{t+j}^o. \quad (5)$$

During this simulation process, our agent makes decisions based on the predicted opponent actions, i.e., $a_t^i \sim \pi(\cdot | \mathbf{o}_t^i, \tilde{a}_t^i \sim \tilde{\pi}_0^o(\cdot | \mathbf{o}_t^o; \phi_0); \theta)$, while the environment model provides a dynamic description of the environment state transitions, i.e., $\mathbf{o}_{t+1}, \mathbf{r}_t = \Gamma(\mathbf{o}_t, \mathbf{a}_t; \zeta)$. It is worth noting that the selection of the k value represents a trade-off between time cost and prediction accuracy, which is not deeply explored in this paper.

After obtaining the simulated data $\{\langle \mathbf{o}^o, \mathbf{a}^{o*} \rangle\}$, we further fine-tune the level-0 opponent model $\tilde{\pi}_0^o(\cdot | \mathbf{o}^o; \phi_0)$ using this data, thereby constructing the level-1 opponent model $\tilde{\pi}_1^o(\cdot | \mathbf{o}^o; \phi_1)$. Our agent can also make behavior choices based on the predictions of the level-1 opponent model, and the opposing agent can further learn from this basis. By recursively iterating the above process, we can generate the level- m opponent model $\tilde{\pi}_m^o(\cdot | \mathbf{o}^o; \phi_m)$.

Through recursive reasoning, we construct M hierarchical opponent models, which are distinct yet interconnected. Clearly, relying on a single opponent model for action prediction is insufficient to fully reflect the opponent's behavior. Therefore, to enhance the representational and adaptive capabilities of the opponent models, we adopt a weighted summation approach to integrate multiple opponent models, forming the final mixed opponent model:

$$\tilde{\pi}_{\text{mix}}^o(\cdot | \mathbf{o}^o) = \sum_{m=0}^{M-1} \kappa_m \tilde{\pi}_m^o(\cdot | \mathbf{o}^o; \phi_m) \quad (6)$$

where κ_m denotes the weight of the level m opponent model, reflecting the probability that the action $\mathbf{a}^o$ is generated by the level m opponent model, i.e., $p(\tilde{\pi}_m^o | \mathbf{a}^o)$, abbreviated as $p(m | \mathbf{a}^o)$. The calculation of the weight $p(m | \mathbf{a}^o)$ can be derived from Bayes' theorem:

$$\begin{aligned} p(m | \mathbf{a}^o) &= \frac{p(\mathbf{a}^o | m) p(m)}{\sum_{i=0}^{M-1} p(\mathbf{a}^o | i) p(i)} \\ &= \frac{\tilde{\pi}_m^o(\mathbf{a}^o | \mathbf{o}^o; \phi_m) p(m)}{\sum_{i=0}^{M-1} [\tilde{\pi}_i^o(\mathbf{a}^o | \mathbf{o}^o; \phi_i) p(i)]}. \end{aligned} \quad (7)$$

Since the prior probability $p(m)$ is difficult to obtain directly, in practice, we use the moving average of $p(m | \mathbf{a}^o)$ to approximate $p(m)$. The calculated decaying moving average $p(m | \mathbf{a}^o)$ is input as Ψ_m into the softer-softmax mixer to obtain the final weight vector κ :

$$\kappa = (\kappa_0, \dots, \kappa_{M-1}) = \text{softer-softmax}(\Psi_0, \dots, \Psi_{M-1}). \quad (8)$$

The softer-softmax [16] mixer is a smoothing operation on probability distributions that allows for overlap between distributions, thereby enhancing the robustness of the model when predicting opponent behavior. The reason for using softer-softmax is that it maintains model diversity while avoiding overfitting to a single opponent model, thereby improving the model's generalization ability in complex adversarial environments. Moreover, according to [39], $M \geq 2$ is robust. Larger M increases the representation capability of the opponent model, but does not always improve the performance, i.e., the gain is vanishing when M increases due to the compounding error. Thus we set $M = 2$ for simplicity.

4.2 Multi-Agent Cooperative Learning with Opponent Modeling

Based on the CTDE paradigm, this paper extends the SAC (Soft Actor-Critic) [14] algorithm to a multi-agent environment, and by

integrating the opponent modeling module mentioned earlier, we propose the MASAC (Multi-Agent SAC) algorithm with embedded opponent modeling. This algorithm combines the adversarial nature of opponent modeling with the cooperative nature of CTDE-based multi-agent reinforcement learning, allowing for cooperative confrontation methods that adapt to various opponent strategies through learning and training.

Each agent i in the team possesses two network models: a policy function π_i and a value function Q_i . The policy function is commonly referred to as the Actor, while the value function is known as the Critic. During the centralized training phase, the Critic receives observations and actions from all agents in the environment to make value judgments. In the decentralized execution phase, the Actor selects actions based on its local observations. Due to its access to more comprehensive environmental information, the Critic can guide the Actor towards a better strategy, making the learning process of the Actor more stable and efficient. It is worth noting that in multi-agent cooperative problems, this paper assumes that in the decentralized execution phase, agents can obtain observations of the opponents (a realistic assumption). With the help of the opponent modeling module, the actions of the opponents can be predicted based on their observations. These predictions are then inputted into the Actor as additional information, thereby aiding the Actor in making better decisions.

Specific to the learning stage of the MARLeOM algorithm, first, sample K tuples $\{(\mathbf{o}_k, \mathbf{a}_k, \mathbf{r}_k, \mathbf{o}_{k+1})\}_{k=1, \dots, K}$ from the replay buffer. For each tuple, compute the target value for agent i using the target network:

$$y_k^i = r_k^i + \gamma \min_{j=1,2} Q^j(\mathbf{o}_{k+1}, \mathbf{a}_{k+1}; \bar{w}_j^i) - \alpha \log \pi^i(a_{k+1}^i | \mathbf{o}_{k+1}^i, a_{k+1}^o; \theta^i), \quad (9)$$

where $a_{k+1}^i \sim \pi^i(\cdot | \mathbf{o}_{k+1}^i, a_{k+1}^o; \theta^i)$. Next, update the two Critic networks of each teammate agent by minimizing the MSE loss function:

$$\mathcal{L}(w_j^i) = \frac{1}{K} \sum_{k=1}^K (y_k^i - Q^j(\mathbf{o}_{k+1}, \mathbf{a}_{k+1}; w_j^i))^2, \quad j = 1, 2, \quad (10)$$

$$w_j^i \leftarrow w_j^i - \beta_w \cdot \nabla_{w_j^i} \mathcal{L}(w_j^i), \quad j = 1, 2,$$

update the Actor network of each teammate agent by maximizing the objective function:

$$J(\theta^i) = \frac{1}{K} \sum_{k=1}^K \left(\min_{j=1,2} Q^j(\mathbf{o}_k, \mathbf{a}_k; w_j^i) - \alpha \log \pi^i(a_k^i | \mathbf{o}_k^i, a_k^o; \theta^i) \right)$$

$$\theta^i \leftarrow \theta^i + \beta_\theta \cdot \nabla_{\theta^i} J(\theta^i), \quad (11)$$

finally, update the target network of each teammate agent:

$$\bar{w}_j^i \leftarrow (1 - \tau) \cdot \bar{w}_j^i + \tau \cdot w_j^i, \quad j = 1, 2. \quad (12)$$

The pseudocode of MARLeOM is given in Algorithm 1.

5 Experiments

In this section, we will illustrate the strength and effectiveness of our algorithm in the Combat² and MPE³ as compared with several state-of-the-art methods. Ablation studies are provided for a better understanding of our method.

Algorithm 1 Multi-Agent Reinforcement Learning Embedded with Opponent Modeling (MARLeOM)

Input: Observations of all agents

Output: Policy of teammate agents

- 1: Initialize Actor and Critic networks for each teammate agent randomly
 - 2: Initialize opponent model weights $\kappa = (1/2, 1/2)$
 - 3: Explore stochastically to collect experience replay buffer $\mathcal{D}$
 - 4: Train first-level opponent model ϕ_0 and environment model Γ_ζ using $\mathcal{D}$
 - 5: Use rollout algorithm within Γ_ζ to obtain opponent's best response a^{o*}
 - 6: Fine-tune ϕ_0 with simulated data $\{ \langle o^o, a^{o*} \rangle \}$ to obtain second-level opponent model ϕ_1
 - 7: Update opponent model weights κ using Bayes' theorem to get mixed opponent model $\tilde{\pi}_{\text{mix}}^o$
 - 8: **for** episode $e = 1 \rightarrow E$ **do**
 - 9: Initialize observations for all agents
 - 10: **for** timestep $t = 1 \rightarrow T$ **do**
 - 11: Predict the opponent action $\tilde{a}_t^o$ using $\tilde{\pi}_{\text{mix}}^o(\cdot | \mathbf{o}_t^o; \phi)$
 - 12: **for** each teammate agent $i = 1 \rightarrow n$ **do**
 - 13: Choose an action a_t^i according to the Actor network $\pi_i(\cdot | \mathbf{o}_t^i, \tilde{a}_t^o; \theta_i)$
 - 14: **end for**
 - 15: The opponent agent selects action a_t^o
 - 16: Refine the first-level opponent model ϕ_0 using the actual action a_t^o chosen by the opponent agent
 - 17: Replicate the steps previously performed within the environmental model to obtain a new mixed opponent model $\tilde{\pi}_{\text{mix}}^o$
 - 18: Execute the joint action and interact with the environment
 - 19: Obtain the next observation $\mathbf{o}_{t+1}$ and reward $\mathbf{r}_{t+1}$ from the environment
 - 20: Store the experience $(\mathbf{o}_t, \mathbf{a}_t, \mathbf{r}_{t+1}, \mathbf{o}_{t+1})$ into $\mathcal{D}$
 - 21: **if** fixed-steps update **then**
 - 22: **for** each teammate agent $i = 1 \rightarrow n$ **do**
 - 23: Sample from $\mathcal{D}$, centralize Critic training, update Actor and target Critic
 - 24: **end for**
 - 25: **end if**
 - 26: **end for**
 - 27: Update Γ_ζ using $\mathcal{D}$
 - 28: **end for**
 - 29: **return** Policy of teammate agents
-

5.1 Multi-Agent Environments

Combat. Combat is a 15×15 grid-based combat environment where two teams of five agents engage in confrontation. Each agent can perform actions such as moving, attacking, or standing by. An attack reduces the target agent's health by 1 point, but it requires a one-second cooldown period. All agents start with 3 health points, and the condition for death is reaching 0 health points. A team wins by eliminating all opposing agents. If no winner is decided within 40 steps, the match is declared a draw. The model is trained to control one team, while the opposing team consists of agents with predefined strategies. Each agent is represented by an ID, team ID, position, health points, and cooldown status. The model receives a reward of -1 or -0.1 multiplied by the sum of the enemy's health points based on the outcome.

² <https://github.com/koulunurag/ma-gym>

³ <https://github.com/openai/multiagent-particle-envs>

Physical deception. The Physical Deception sub-environment is specifically designed for the study of deceptive behaviors and strategies. Within this environment, there typically exist two roles: the Deceiver and the Target. The objective of the Target agent is to avoid the Deceiver as much as possible, usually by moving within the environment. The Deceiver’s goal is to approach the Target; however, due to certain physical constraints (e.g., slower speed), it must employ deceptive strategies to achieve its aim. The Deceiver may approach the Target by mimicking the Target’s movement patterns or by taking misleading actions. The environment is a two-dimensional space where agents can move freely. The Deceiver may have specific deceptive actions or strategies, which require learning for optimization. The reward function is typically designed to incentivize the Deceiver to successfully approach the Target, while penalizing the Target for being captured. For instance, when the distance between the Deceiver and the Target is less than a certain threshold, the Deceiver receives a positive reward, whereas the Target incurs a negative reward.

Predator-prey. The Predator-Prey sub-environment is a classic setting that simulates the predator-prey dynamics, typically involving multiple predator and prey agents. The objective of the predators is to capture the prey. In many cases, predators may need to cooperate with other predators to enhance the success rate of capturing prey. The prey’s goal is to evade the predators as effectively as possible. Prey may develop evasion strategies, such as hiding or moving rapidly. The environment is also a two-dimensional space where agents can navigate. Predators and prey usually have different capabilities for observation and action. Predators receive positive rewards when they successfully capture prey, while prey incur negative rewards upon being captured. Conversely, prey receive positive rewards for successfully evading predators.



Figure 2: Illustrations of some environments we consider, including *Combat* (left), *Physical Deception* (middle) and *Predator-Prey* (right).

5.2 Experiment Settings & Baselines

In conducting comparative experiments, we selected the following baseline algorithms: IPPO [9], VDN [29], MADDPG [24], COMA [11], QMIX [27], and MAPPO [38]. These algorithms encompass the fundamental, canonical, and widely-employed paradigms within the field of multi-agent reinforcement learning. The experimental setup was consistent with that described in [24]. In the Combat environment, adversaries adhere to a predefined set of rules; accordingly, we utilized this environment to evaluate the performance against adversaries with fixed regulations. In contrast, the Physical Deception and Predator-Prey scenarios within the MPE framework permit the modification of reward structures and combat strategies, where we conducted assessments against both conventional and disruptive adversaries. Furthermore, to efficiently validate the efficacy of the opponent modeling module, we conducted ablation experiments within the Combat environment. Regarding evaluation metrics, the Combat environment employs the win rate, defined as the proportion of games in which the agent emerges victorious over its opponent; whereas, the MPE environment utilizes the

rewards (scores) accumulated by the agents as the criterion for assessment.

5.3 Evaluation with Fixed-Rule Opponents

Figure 3 illustrates the win rates achieved by agents employing different algorithms when matched against opponents that consistently follow predefined rules throughout the training regimen. The graph clearly demonstrates that the MARLeOM algorithm not only converges more quickly but also attains a significantly higher win rate. This strong performance indicates that the MARLeOM algorithm possesses enhanced learning capabilities when contrasted with the other algorithms under examination. The data underscores the effectiveness of incorporating opponent modeling into the learning process, allowing the MARLeOM algorithm to more adeptly anticipate and counter the fixed strategies employed by its adversaries. This result is particularly noteworthy as it highlights the potential for MARLeOM to outperform traditional reinforcement learning methods in competitive environments where the behavior of opponents is predictable.

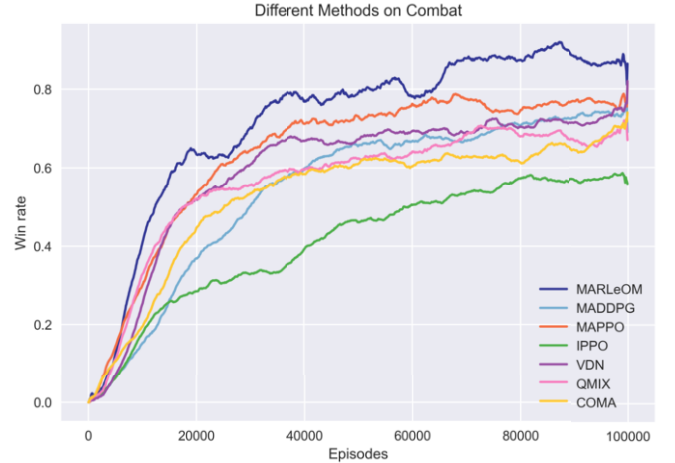


Figure 3: The win rate of the the controlled agents of different algorithms against fixed-rule opponents

5.4 Evaluation with Normal Opponents

This part conducts experimental evaluations against conventional opponents. In this context, a conventional opponent refers to an adversary that can learn and make decisions based on conventional behavioral logic (i.e., choosing actions aimed at maximizing rewards) during the adversarial process. To objectively assess the quality of strategies trained by different algorithms in adversarial environments, the following design was implemented in two MPE environments: agents trained with the MARLeOM algorithm proposed in this chapter were placed in the roles of both ordinary and adversarial agents, facing agents trained with baseline algorithms. This formed four adversarial combinations: MARLeOM versus the baseline algorithm, MARLeOM versus MARLeOM, the baseline algorithm versus MARLeOM, and the baseline algorithm versus the baseline algorithm. The necessity of this design lies in the asymmetry of roles between ordinary and adversarial agents. Each algorithm model was trained until convergence, followed by a performance evaluation over 100 iterations. The evaluation metric was the cumulative reward (score) obtained by the agents, and the average metric was used as the final performance indicator for the agents.

The comparison results of MARLeOM and Baselines in two MPE environments are shown in Figure 4. In this experiment, the rewards of ordinary agents in different scenarios were measured and normalized to the range of 0-1. It can be observed that, in all environments, the highest score was achieved when the MARLeOM agent acted as the ordinary agent against the Baseline agent (MARLeOM VS Baseline); conversely, the lowest score was obtained when Baseline was the ordinary agent against the MARLeOM adversary (Baseline VS MARLeOM). This indicates that the quality of strategies trained by MARLeOM is significantly higher than that of Baselines.

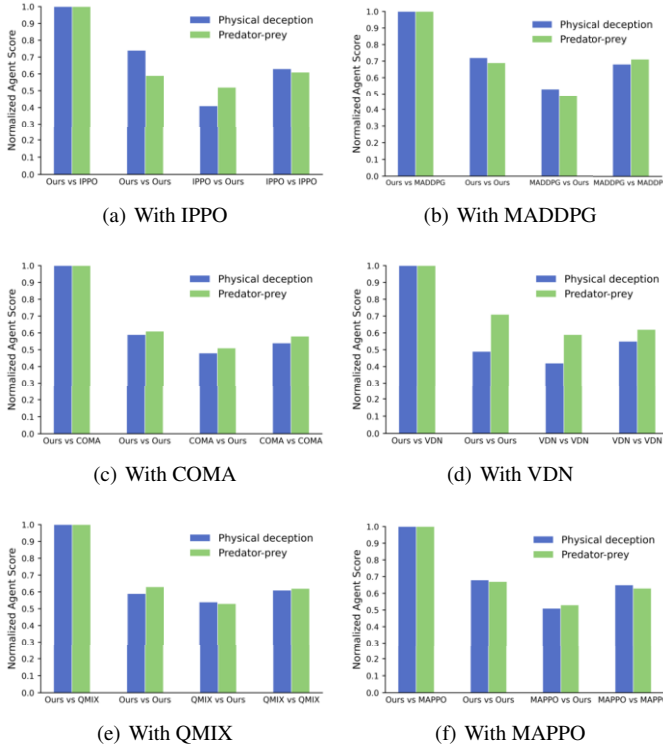


Figure 4: Comparison between MARLeOM and the baseline on the two mixed competitive environments. Each bar cluster shows the 0-1 normalized score for a set of competing policies in different roles (agent VS adversary), where a higher score is better for the agent. In all cases, MARLeOM outperforms all baselines.

5.5 Evaluation with Disruptive Opponents

To evaluate the performance of MARLeOM under highly adverse conditions, we conducted comparative experiments against disruptive adversaries. These adversaries exhibit pronounced targeting and aggressiveness during confrontations. In constructing such adversaries, we initially fixed a pre-trained normal agent (either MARLeOM or a baseline algorithm) and then set the rewards for the disruptive adversary to be the negative of the normal agent's rewards (thereby converting the problem into a zero-sum game). Subsequently, we trained the disruptive adversary from scratch using the SAC algorithm.

This method of constructing disruptive adversaries takes into account the characteristics of the MPE environment. Typically, competitive scenarios within the MPE environment are not zero-sum. This implies that, according to the default reward function, the objectives of the adversary may not align with those of disrupting the normal agent. Consequently, we needed to adjust the rewards within the environment to a zero-sum format. Since there is only one adversary in

the MPE environment, fixing the normal agent simplifies the learning problem to a single-agent setting, in which the SAC algorithm can be reliably employed to train the disruptive adversary.

As illustrated in Figure 5, the experiment depicts the reward variation curves for normal agents fixed by different algorithms during the training process of the disruptive adversary until convergence in the Physical Deception environment. The figure reveals that the MARLeOM agent achieved higher rewards in this scenario, indicating that even under the most unfavorable conditions, MARLeOM demonstrates enhanced robustness. This increased robustness may be attributed to the opponent modeling module within MARLeOM, which is capable of more rapidly and effectively adapting to changes in the adversary's strategies.

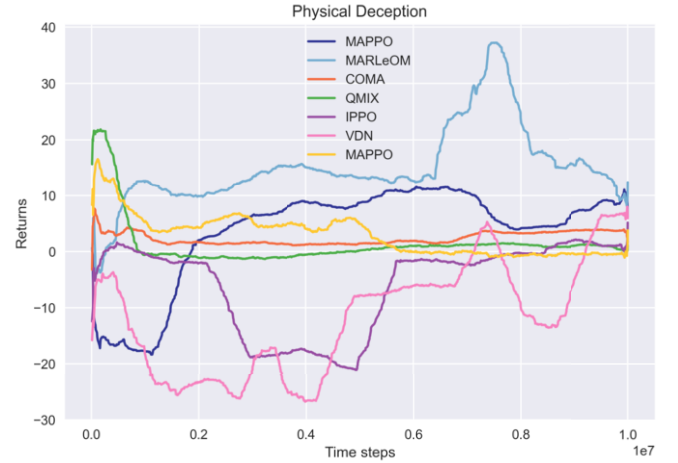


Figure 5: Performances of MARLeOM and baselines under the worst situation, i.e., against the disruptive opponents.

5.6 Ablation Study

In this section we perform an ablation study to justify the design choices behind the architecture of MARLeOM. We design and evaluate four different ablations of MARLeOM in the Combat environment: MARLeOM-0 which has no opponent modeling, MARLeOM-1 which has only one-level opponent model, MARLeOM-3 which has 3 opponent models with different reasoning level and MARLeOM-4 which has 4 opponent models with different reasoning level.

Figure 6 depicts the win rates for the MARLeOM algorithm and the previously mentioned ablation baselines within the Combat environment. From the figure, it is evident that MARLeOM exhibits superior performance compared to MARLeOM-1, which in turn outperforms MARLeOM-0. Additionally, the performance of MARLeOM is found to be closely aligned with that of MARLeOM-3 and MARLeOM-4. These results collectively highlight the effectiveness of the MARLeOM design.

6 Conclusion & Future Work

Addressing the small-scale multi-agent cooperative adversarial problem, we propose a multi-agent reinforcement learning algorithm with embedded opponent modeling that adapts to the dynamic strategies of adversaries. The opponent modeling module of the algorithm constructs multi-level models of adversaries using environmental models and recursive reasoning techniques, enhancing their representational power through the integration of these models. In the part of

multi-agent reinforcement learning, we adopt the paradigm of centralized training with decentralized execution and the Actor-Critic framework. The Actor makes decisions based on local information, which includes its own observations and opponent actions predicted by the opponent modeling module. We have selected multiple multi-agent testing environments as experimental platforms and conducted experimental evaluations against different types of adversaries based on the characteristics of each environment. Comparative experimental results indicate that the proposed algorithm exhibits stronger cooperative adversarial capabilities compared to baseline algorithms. Moreover, ablation experiments on the opponent modeling module further validate the rationality of the algorithm design.

Looking forward, future research will be extended to learning adversarial strategies in large-scale multi-agent systems and to improving the sampling efficiency of environmental models in opponent modeling methods. We are committed to exploring more efficient learning mechanisms to cope with complex and variable environments and adversary strategies, thereby promoting the in-depth development of multi-agent systems in both theoretical and applied dimensions.

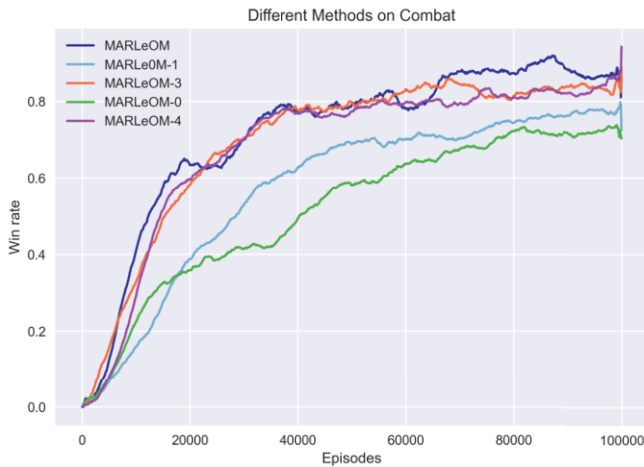


Figure 6: The results of the ablation study

Acknowledgements

This paper was funded by the Humanity and Social Science Youth Research Foundation of Ministry of Education of China (Grant No. 19YJC790111) and the Philosophy and Social Science Post-Foundaion of Ministry of Education of China (Grant No. 18JHQ060).

References

- [1] M. Al-Shedivat, T. Bansal, Y. Burda, I. Sutskever, I. Mordatch, and P. Abbeel. Continuous adaptation via meta-learning in nonstationary and competitive environments. *arXiv preprint arXiv:1710.03641*, 2017.
- [2] Y. Bai and C. Jin. Provable self-play algorithms for competitive reinforcement learning. In *International conference on machine learning*, pages 551–560. PMLR, 2020.
- [3] Y. Bai, C. Jin, and T. Yu. Near-optimal reinforcement learning with self-play. *Advances in neural information processing systems*, 33:2159–2170, 2020.
- [4] R. I. Brafman and M. Tennenholtz. R-max-a general polynomial time algorithm for near-optimal reinforcement learning. *Journal of Machine Learning Research*, 3(Oct):213–231, 2002.
- [5] J. M. Catacora Ocana, F. Riccio, R. Capobianco, and D. Nardi. Cooperative multi-agent deep reinforcement learning in soccer domains. In *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*, pages 1865–1867, 2019.
- [6] C. Claus and C. Boutilier. The dynamics of reinforcement learning in cooperative multiagent systems. *AAAI/IAAI*, 1998(746-752):2, 1998.
- [7] J. Cui, Y. Liu, and A. Nallanathan. Multi-agent reinforcement learning-based resource allocation for uav networks. *IEEE Transactions on Wireless Communications*, 19(2):729–743, 2019.
- [8] Z. Dai, Y. Chen, B. K. H. Low, P. Jaillet, and T.-H. Ho. R2-b2: Recursive reasoning-based bayesian optimization for no-regret learning in games. In *International Conference on Machine Learning*, pages 2291–2301. PMLR, 2020.
- [9] C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. Torr, M. Sun, and S. Whiteson. Is independent learning all you need in the starcraft multi-agent challenge? *arXiv preprint arXiv:2011.09533*, 2020.
- [10] J. Ferber and G. Weiss. *Multi-agent systems: an introduction to distributed artificial intelligence*, volume 1. Addison-wesley Reading, 1999.
- [11] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [12] J. N. Foerster, R. Y. Chen, M. Al-Shedivat, S. Whiteson, P. Abbeel, and I. Mordatch. Learning with opponent-learning awareness. *arXiv preprint arXiv:1709.04326*, 2017.
- [13] A. Grover, M. Al-Shedivat, J. Gupta, Y. Burda, and H. Edwards. Learning policy representations in multiagent systems. In *International conference on machine learning*, pages 1802–1811. PMLR, 2018.
- [14] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.
- [15] H. He, J. Boyd-Graber, K. Kwok, and H. Daumé III. Opponent modeling in deep reinforcement learning. In *International conference on machine learning*, pages 1804–1813. PMLR, 2016.
- [16] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [17] Z.-W. Hong, S.-Y. Su, T.-Y. Shann, Y.-H. Chang, and C.-Y. Lee. A deep policy inference q-network for multi-agent systems. *arXiv preprint arXiv:1712.07893*, 2017.
- [18] J. Hu and M. P. Wellman. Nash q-learning for general-sum stochastic games. *Journal of machine learning research*, 4(Nov):1039–1069, 2003.
- [19] C. Jin, Q. Liu, and T. Yu. The power of exploiter: Provable multi-agent rl in large state spaces. In *International Conference on Machine Learning*, pages 10251–10279. PMLR, 2022.
- [20] D. K. Kim, M. Liu, M. D. Riemer, C. Sun, M. Abdulhai, G. Habibi, S. Lopez-Cot, G. Tesauro, and J. How. A policy gradient algorithm for learning to learn in multiagent reinforcement learning. In *International Conference on Machine Learning*, pages 5541–5550. PMLR, 2021.
- [21] M. L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*, pages 157–163. Elsevier, 1994.
- [22] Q. Liu, T. Yu, Y. Bai, and C. Jin. A sharp analysis of model-based reinforcement learning with self-play. In *International Conference on Machine Learning*, pages 7001–7010. PMLR, 2021.
- [23] X. Liu, J. Yu, Z. Feng, and Y. Gao. Multi-agent reinforcement learning for resource allocation in iot networks with edge computing. *China Communications*, 17(9):220–236, 2020.
- [24] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems*, 30, 2017.
- [25] N. Rabinowitz, F. Perbet, F. Song, C. Zhang, S. A. Eslami, and M. Botvinick. Machine theory of mind. In *International conference on machine learning*, pages 4218–4227. PMLR, 2018.
- [26] R. Raileanu, E. Denton, A. Szlam, and R. Fergus. Modeling others using oneself in multi-agent reinforcement learning. In *International conference on machine learning*, pages 4257–4266. PMLR, 2018.
- [27] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *The Journal of Machine Learning Research*, 21(1):7234–7284, 2020.
- [28] F. M. Rocha, V. S. Costa, and L. P. Reis. From reinforcement learning towards artificial general intelligence. *Trends and Innovations in Information Systems and Technologies: Volume 2* 8, pages 401–413, 2020.
- [29] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296*, 2017.
- [30] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*.

- 2018.
- [31] G. Tesauro and G. Galperin. On-line policy improvement using monte-carlo search. *Advances in Neural Information Processing Systems*, 9, 1996.
 - [32] Y. Tian, Y. Wang, T. Yu, and S. Sra. Online learning in unknown markov games. In *International conference on machine learning*, pages 10279–10288. PMLR, 2021.
 - [33] C.-Y. Wei, Y.-T. Hong, and C.-J. Lu. Online reinforcement learning in stochastic games. *Advances in Neural Information Processing Systems*, 30, 2017.
 - [34] C.-Y. Wei, C.-W. Lee, M. Zhang, and H. Luo. Linear last-iterate convergence in constrained saddle-point optimization. *arXiv preprint arXiv:2006.09517*, 2020.
 - [35] Y. Wen, Y. Yang, R. Luo, and J. Wang. Modelling bounded rationality in multi-agent interactions by generalized recursive reasoning. *arXiv preprint arXiv:1901.09216*, 2019.
 - [36] Y. Wen, Y. Yang, R. Luo, J. Wang, and W. Pan. Probabilistic recursive reasoning for multi-agent reinforcement learning. *arXiv preprint arXiv:1901.09207*, 2019.
 - [37] Q. Xie, Y. Chen, Z. Wang, and Z. Yang. Learning zero-sum simultaneous-move markov games using function approximation and correlated equilibrium. In *Conference on learning theory*, pages 3674–3682. PMLR, 2020.
 - [38] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35:24611–24624, 2022.
 - [39] X. Yu, J. Jiang, W. Zhang, H. Jiang, and Z. Lu. Model-based opponent modeling. *Advances in Neural Information Processing Systems*, 35: 28208–28221, 2022.